

Correction — TP « De l'arbre à la forêt »

Création du jeu de données

Le code fourni permet d'obtenir :

- 1 000 infrastructures ;
- 5 variables numériques ;
- 2 variables catégorielles ;
- une cible binaire 0 / 1.

La cible est créée à partir de :

```
âge de l'infrastructure  
+ nombre d'incidents  
+ retard de maintenance  
+ exposition aux intempéries  
+ bruit aléatoire
```

Les variables `traffic_level`, `infrastructure_type` et `region` ne participent **pas directement** à la génération de la cible.

C'est particulièrement intéressant pour le challenge sur l'importance des variables.

```
import matplotlib.pyplot as plt  
from sklearn.datasets import make_classification  
from sklearn.ensemble import RandomForestClassifier  
from sklearn.model_selection import train_test_split  
from sklearn.metrics import accuracy_score
```

```
import numpy as np  
import pandas as pd  
  
rng = np.random.default_rng(42)  
n_samples = 1000  
  
X = pd.DataFrame({  
    'infrastructure_age': rng.integers(1, 60, n_samples),  
    'annual_incidents': rng.poisson(2, n_samples),  
    'maintenance_delay': rng.integers(0, 120, n_samples),  
    'weather_exposure': rng.uniform(0, 1, n_samples),  
    'traffic_level': rng.integers(500, 15000, n_samples),  
    'infrastructure_type': rng.choice(['bridge', 'road', 'tunnel'], n_samples),
```

```

    'region': rng.choice(['north', 'south', 'east', 'west'], n_samples)
})

# Génération de la cible avec une règle métier complexe et un peu de bruit
risk_score = (
    (X['infrastructure_age'] / 60) * 0.3 +
    (X['annual_incidents'] / 5) * 0.3 +
    (X['maintenance_delay'] / 120) * 0.2 +
    (X['weather_exposure']) * 0.2
)
# Ajout d'une part de hasard
risk_score += rng.normal(0, 0.1, n_samples)
y = (risk_score > 0.6).astype(int)

```

Niveau 1 — Guidé

Partie 1 — Découverte des données

Correction

```
print(X.head())
```

	infrastructure_age	annual_incidents	maintenance_delay	weather_exposure	\
0	6	5	71	0.902370	
1	46	2	45	0.796467	
2	39	0	95	0.946892	
3	26	0	60	0.651962	
4	26	1	118	0.662922	

	traffic_level	infrastructure_type	region
0	7283	road	east
1	9627	tunnel	west
2	5187	bridge	north
3	9707	bridge	west
4	5230	tunnel	north

On peut ensuite identifier les types :

```
print(X.dtypes)
```

```
infrastructure_age      int64
annual_incidents        int64
maintenance_delay      int64
weather_exposure       float64
traffic_level          int64
infrastructure_type     object
region                 object
dtype: object
```

Variables numériques :

```
numeric_features = [
    'infrastructure_age',
    'annual_incidents',
    'maintenance_delay',
    'weather_exposure',
    'traffic_level'
]
```

Variables catégorielles :

```
categorical_features = [
    'infrastructure_type',
    'region'
]
```

Distribution de la cible

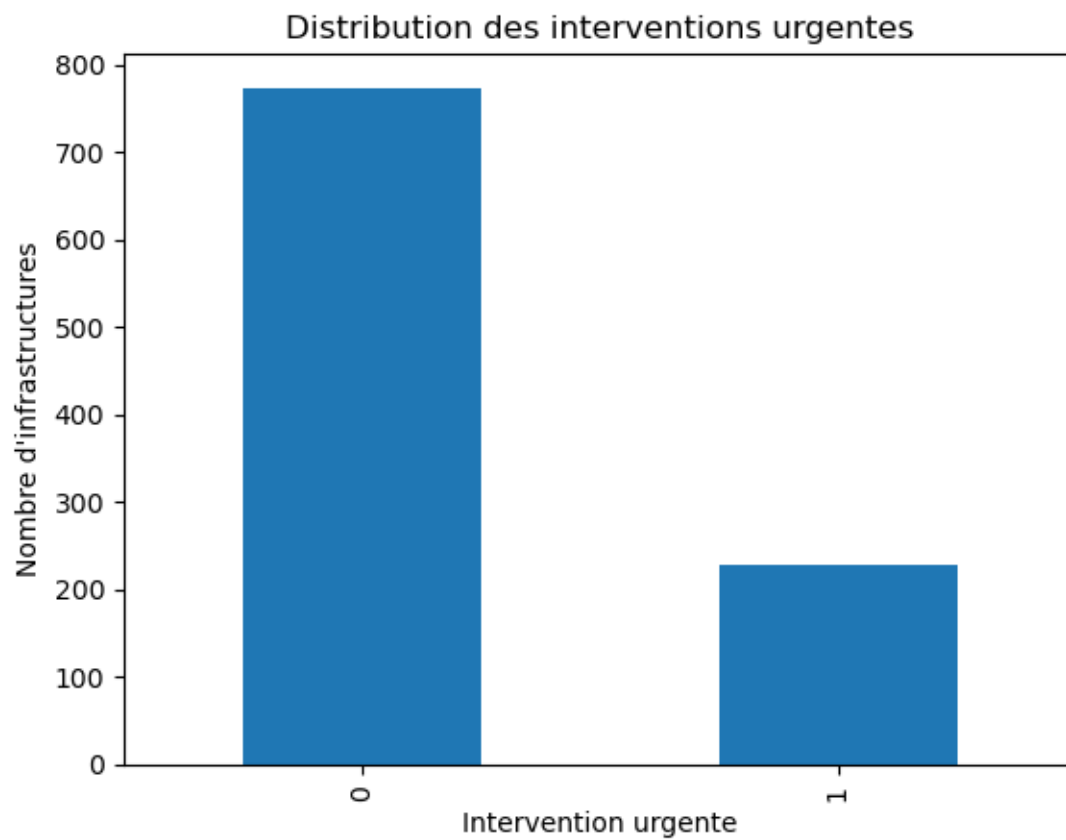
```
print(y.value_counts())
print(y.value_counts(normalize=True))
```

```
0    773
1    227
Name: count, dtype: int64
0    0.773
1    0.227
Name: proportion, dtype: float64
```

On peut également visualiser :

```
y.value_counts().plot(
    kind='bar',
    title='Distribution des interventions urgentes'
)

plt.xlabel('Intervention urgente')
plt.ylabel('Nombre d\'infrastructures')
plt.show()
```



Valeurs manquantes

```
print(X.isna().sum())
```

```
infrastructure_age      0
annual_incidents        0
maintenance_delay       0
weather_exposure        0
traffic_level           0
infrastructure_type      0
region                  0
dtype: int64
```

Résultat attendu :

```
infrastructure_age      0
annual_incidents        0
maintenance_delay       0
weather_exposure        0
traffic_level           0
infrastructure_type      0
region                  0
dtype: int64
```

Réponses

1. Quelle est la variable cible ?

La variable cible est y.

Elle indique :

- 0 → aucune intervention urgente prévue ;
- 1 → intervention urgente prévue.

2. Les classes sont-elles équilibrées ?

Elles ne sont probablement pas parfaitement équilibrées.

Le bon réflexe est cependant de **ne pas répondre sans regarder les proportions**.

Partie 2 — Construire un premier arbre

Construction du preprocessing

La première étape consiste à séparer les variables numériques et catégorielles.

```

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

preprocessor = ColumnTransformer(
    transformers=[
        (
            'categorical',
            OneHotEncoder(handle_unknown='ignore'),
            categorical_features
        ),
        (
            'numeric',
            'passthrough',
            numeric_features
        )
    ]
)

```

On peut ensuite construire l'arbre :

```

from sklearn.pipeline import Pipeline
from sklearn.tree import DecisionTreeClassifier

decision_tree = Pipeline([
    ('preprocessor', preprocessor),
    (
        'model',
        DecisionTreeClassifier(
            max_depth=None,
            random_state=42
        )
    )
])

```

Puis :

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

```

```

decision_tree.fit(X_train, y_train)

print(

```

```

    f"Train : {decision_tree.score(X_train, y_train):.3f}"
)

print(
    f"Test : {decision_tree.score(X_test, y_test):.3f}"
)

```

```

Train : 1.000
Test : 0.800

```

```

Train : 1.000
Test : ~0.80

```

La valeur exacte du score de test dépend de la génération des données et de la version de scikit-learn.

Réponse attendue

Pourquoi les performances sur les données d'entraînement sont-elles très supérieures à celles sur les données de test ?

L'arbre est suffisamment complexe pour **apprendre très précisément les données d'entraînement**.

Un arbre sans contrainte de profondeur peut continuer à créer des séparations jusqu'à presque parfaitement séparer les observations.

Il risque alors de mémoriser une partie du bruit présent dans les données.

C'est le phénomène de **surapprentissage (overfitting)**.

On observe donc généralement :

```

Performance entraînement
    ↓
    très élevée

Performance test
    ↓
    plus faible

```

Partie 3 — Construire une Random Forest

La forêt reprend exactement le même preprocessing.

```
from sklearn.ensemble import RandomForestClassifier

random_forest = Pipeline([
    ('preprocessor', preprocessor),
    (
        'model',
        RandomForestClassifier(
            n_estimators=100,
            random_state=42
        )
    )
])

random_forest.fit(X_train, y_train)
```

```
Pipeline(steps=[('preprocessor',
                  ColumnTransformer(transformers=[('categorical',
                                                  OneHotEncoder(handle_unknown='ignore'),
                                                  ['infrastructure_type',
                                                  'region']),
                                                  ('numeric', 'passthrough',
                                                  ['infrastructure_age',
                                                  'annual_incidents',
                                                  'maintenance_delay',
                                                  'weather_exposure',
                                                  'traffic_level'])])),
                ('model', RandomForestClassifier(random_state=42))])
```

On récupère les scores :

```
results = pd.DataFrame({
    'model': [
        'Decision Tree',
        'Random Forest'
    ],
    'train_score': [
        decision_tree.score(X_train, y_train),
```

```

        random_forest.score(X_train, y_train)
    ],
    'test_score': [
        decision_tree.score(X_test, y_test),
        random_forest.score(X_test, y_test)
    ]
})

results

```

	model	train_score	test_score
0	Decision Tree	1.0	0.800000
1	Random Forest	1.0	0.846667

Interprétation attendue

Typiquement :

Modèle	Train	Test
Decision Tree	très élevé	plus faible
Random Forest	très élevé	élevé

La forêt peut également avoir un score d'entraînement inférieur à celui de l'arbre individuel.

Ce n'est **pas un problème**.

L'objectif n'est pas d'obtenir le meilleur score possible sur les données d'entraînement.

L'objectif est d'obtenir un modèle qui **généralise correctement sur de nouvelles infrastructures**.

Partie 4 — Observer les arbres de la forêt

On récupère la forêt :

```

forest = random_forest.named_steps['model']

print(f'Nombre d\'arbres : {len(forest.estimators_)}')

```

Nombre d'arbres : 100

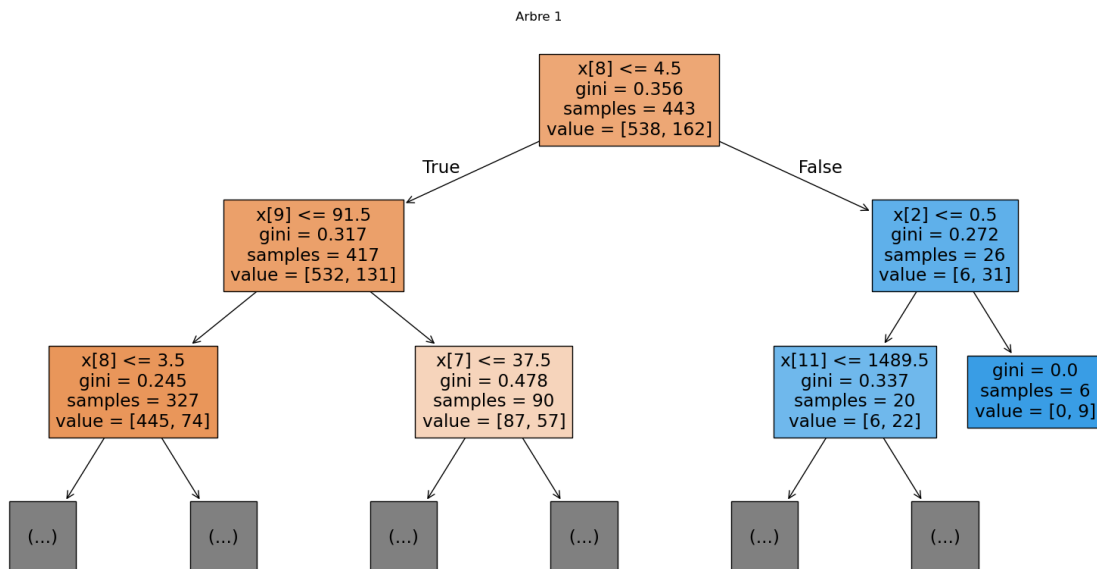
On visualise le premier arbre :

```
from sklearn.tree import plot_tree

plt.figure(figsize=(20, 10))

plot_tree(
    forest.estimators_[0],
    max_depth=2,
    filled=True
)

plt.title('Arbre 1')
plt.show()
```



Puis le deuxième :

```
plt.figure(figsize=(20, 10))

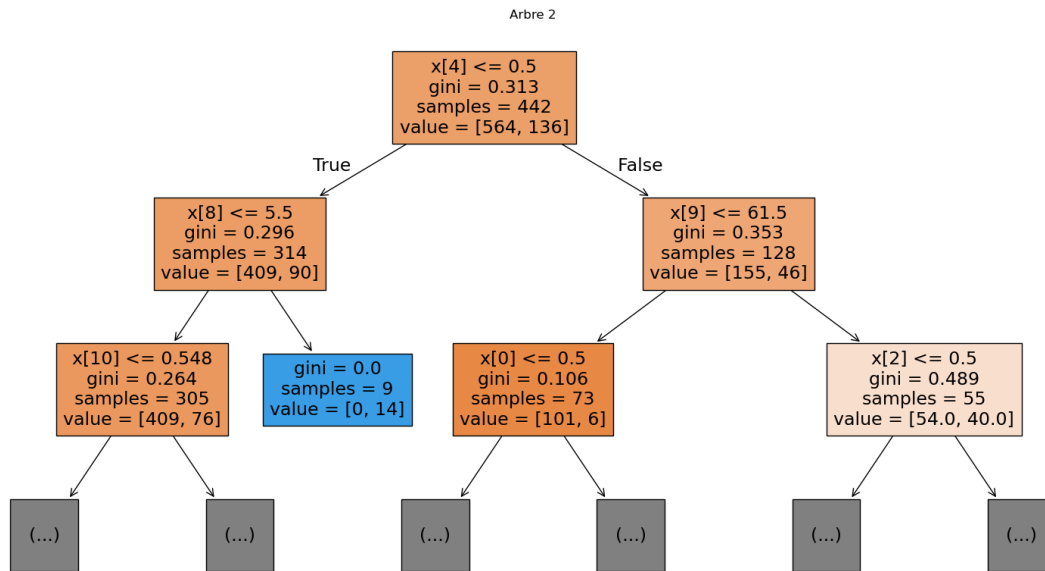
plot_tree(
    forest.estimators_[1],
    max_depth=2,
```

```

filled=True
)

plt.title('Arbre 2')
plt.show()

```



Réponses

1. Les deux arbres sont-ils identiques ?

Non.

Ils ont généralement des structures et des règles de séparation différentes.

2. Pourquoi ?

Parce que Random Forest introduit de la **diversité** entre les arbres.

Deux mécanismes sont particulièrement importants :

1. Bootstrap

Chaque arbre est entraîné sur un échantillon différent des données d'entraînement.

2. Sélection aléatoire des variables

À chaque séparation, l'arbre ne considère qu'un sous-ensemble des variables disponibles.

On obtient donc :

Dataset

```
→ Arbre 1
    données différentes
    variables différentes

→ Arbre 2
    données différentes
    variables différentes

→ Arbre 3

→ ...
```

C'est cette diversité qui permet ensuite de faire travailler les arbres **ensemble**.

Partie 5 — Faire voter la forêt

Le code fourni permet d'obtenir les votes :

```
sample = X_test.iloc[[0]]

sample_preprocessed = (
    random_forest
    .named_steps['preprocessor']
    .transform(sample)
)

predictions_arbres = []

for tree in forest.estimators_:
    pred = tree.predict(sample_preprocessed)
    predictions_arbres.append(pred[0])

print(predictions_arbres[:10])
```

```
[np.float64(0.0), np.float64(0.0), np.float64(0.0), np.float64(0.0), np.float64(0.0), np.float64(0.0)]
```

On peut compter les votes :

```
from collections import Counter
votes = Counter(predictions_arbres)
print(votes)
```

```
Counter({np.float64(0.0): 95, np.float64(1.0): 5})
```

Par exemple :

```
Counter({0: 73, 1: 27})
```

La majorité a donc voté 0.

On peut reproduire la décision :

```
final_prediction = max(
    votes,
    key=votes.get
)

print(final_prediction)
```

0.0

Et comparer avec scikit-learn :

```
print(random_forest.predict(sample))
```

[0]

Réponse attendue

La Random Forest effectue une **majorité des votes**.

Par exemple :

73 arbres → 0

27 arbres → 1

↓

Prédiction finale → 0

Niveau 2 — Réflexion

Partie 6 — Combien d'arbres faut-il ?

Je conseillerais de stocker les résultats dans une liste.

```
n_estimators_values = [
    1, 5, 10, 25, 50, 100, 200, 500
]

results = []

for n_estimators in n_estimators_values:

    model = Pipeline([
        ('preprocessor', preprocessor),
        (
            'model',
            RandomForestClassifier(
                n_estimators=n_estimators,
                random_state=42
            )
        )
    ])

    model.fit(X_train, y_train)

    results.append({
        'n_estimators': n_estimators,
        'train_score': model.score(X_train, y_train),
        'test_score': model.score(X_test, y_test)
    })
```

```
results_df = pd.DataFrame(results)
```

```
results_df
```

	n_estimators	train_score	test_score
0	1	0.928571	0.790000
1	5	0.977143	0.823333
2	10	0.988571	0.833333
3	25	1.000000	0.846667
4	50	1.000000	0.846667
5	100	1.000000	0.846667
6	200	1.000000	0.843333
7	500	1.000000	0.836667

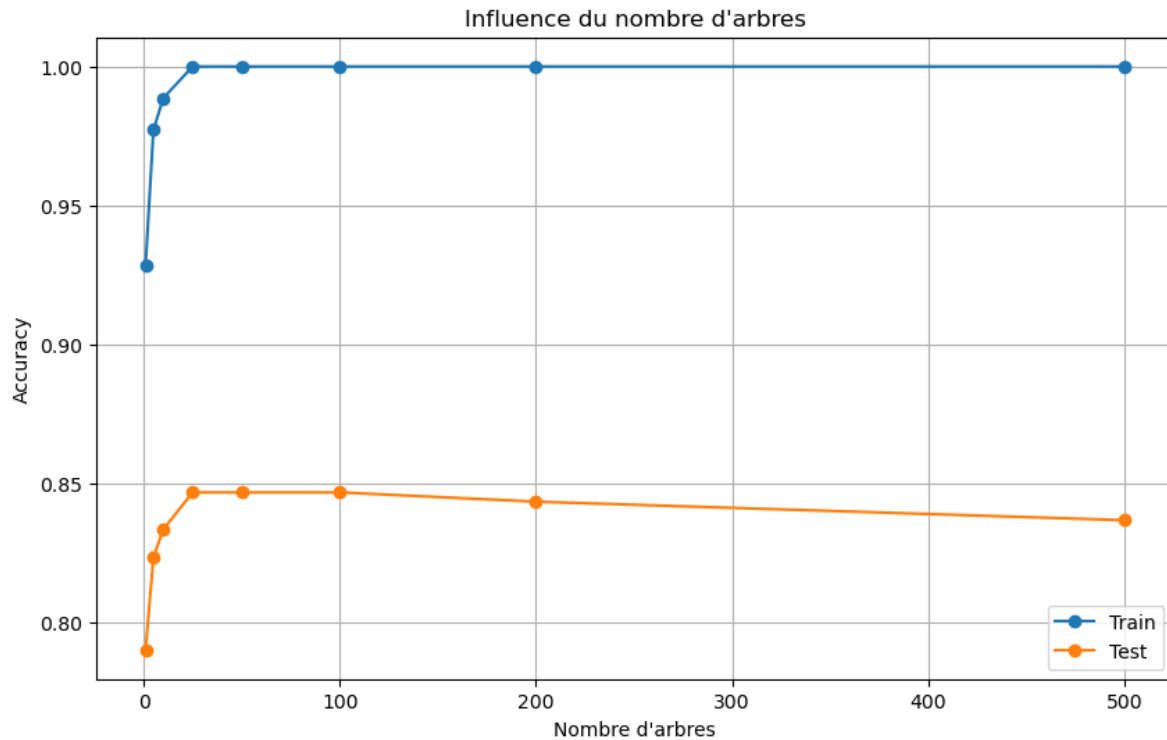
Puis :

```
plt.figure(figsize=(10, 6))

plt.plot(
    results_df['n_estimators'],
    results_df['train_score'],
    marker='o',
    label='Train'
)

plt.plot(
    results_df['n_estimators'],
    results_df['test_score'],
    marker='o',
    label='Test'
)

plt.xlabel("Nombre d'arbres")
plt.ylabel('Accuracy')
plt.title("Influence du nombre d'arbres")
plt.legend()
plt.grid()
plt.show()
```



Réponse 1

Que se passe-t-il après 50 arbres ?

La performance tend généralement à **se stabiliser**.

Ajouter davantage d'arbres peut encore améliorer légèrement les résultats, mais les gains deviennent progressivement faibles.

On peut représenter l'idée ainsi :

Performance

1 10 25 50 100 500
arbres

Réponse 2

Pourquoi ne pas utiliser 10 000 arbres ?

Parce que le coût computationnel augmente avec le nombre d'arbres.

Si passer de 100 à 200 arbres n'apporte quasiment aucune amélioration, entraîner 10 000 arbres n'est généralement pas justifié.

Il existe donc un **compromis entre performance et coût de calcul**.

Partie 7 — Le rôle de `max_features`

On peut automatiser le test :

```
max_features_values = [
    'sqrt',
    'log2',
    None
]

results = []

for max_features in max_features_values:

    model = Pipeline([
        ('preprocessor', preprocessor),
        (
            'model',
            RandomForestClassifier(
                n_estimators=100,
                max_features=max_features,
                random_state=42
            )
        )
    ])

    model.fit(x_train, y_train)

    results.append({
        'max_features': str(max_features),
        'train_score': model.score(x_train, y_train),
```

```
        'test_score': model.score(x_test, y_test)
    })

pd.DataFrame(results)
```

Explication

`max_features` agit sur la **diversité des arbres**.

Avec :

```
max_features=None
```

chaque arbre peut utiliser toutes les variables disponibles à chaque séparation.

Les arbres risquent donc de devenir davantage similaires.

Avec :

```
max_features='sqrt'
```

chaque séparation ne considère qu'une partie des variables.

On augmente ainsi la diversité des arbres.

Réponse attendue

La force d'une Random Forest ne vient pas simplement du fait qu'elle possède beaucoup d'arbres.

Elle vient de :

beaucoup d'arbres + suffisamment différents les uns des autres.

Si tous les arbres font exactement les mêmes erreurs, les faire voter ensemble n'apporte presque rien.

Partie 8 — La forêt contre l'arbre

On peut construire une boucle pour les deux modèles :

```

max_depth_values = [
    2,
    4,
    6,
    10,
    None
]

results = []

for max_depth in max_depth_values:

    tree = Pipeline([
        ('preprocessor', preprocessor),
        (
            'model',
            DecisionTreeClassifier(
                max_depth=max_depth,
                random_state=42
            )
        )
    ])

    forest = Pipeline([
        ('preprocessor', preprocessor),
        (
            'model',
            RandomForestClassifier(
                n_estimators=100,
                max_depth=max_depth,
                random_state=42
            )
        )
    ])

    tree.fit(x_train, y_train)
    forest.fit(x_train, y_train)

    results.append({
        'model': 'Decision Tree',
        'max_depth': str(max_depth),
        'train_score': tree.score(x_train, y_train),

```

```

        'test_score': tree.score(x_test, y_test)
    })

    results.append({
        'model': 'Random Forest',
        'max_depth': str(max_depth),
        'train_score': forest.score(x_train, y_train),
        'test_score': forest.score(x_test, y_test)
    })

depth_results = pd.DataFrame(results)

depth_results

```

On peut ensuite tracer :

```

for model_name in ['Decision Tree', 'Random Forest']:

    subset = depth_results[
        depth_results['model'] == model_name
    ]

    plt.figure(figsize=(10, 6))

    plt.plot(
        subset['max_depth'],
        subset['train_score'],
        marker='o',
        label='Train'
    )

    plt.plot(
        subset['max_depth'],
        subset['test_score'],
        marker='o',
        label='Test'
    )

    plt.title(model_name)
    plt.xlabel('Max depth')
    plt.ylabel('Accuracy')
    plt.legend()

```

```
plt.grid()
plt.show()
```

Ce qu'on cherche à faire observer

Avec un arbre :

```
petite profondeur
  ↓
modèle trop simple
  ↓
sous-apprentissage
```

Puis :

```
profondeur ↑
  ↓
modèle plus complexe
  ↓
meilleure performance
```

Puis éventuellement :

```
profondeur excessive
  ↓
surapprentissage
```

La Random Forest est généralement **plus robuste** à cette variation grâce à l'agrégation de nombreux arbres.

Niveau 3 — Challenge

Challenge 1 — Trouver la meilleure forêt

Je déconseille de donner une « meilleure combinaison » à l'étudiant avant qu'il cherche.

Pour la correction, on peut proposer par exemple :

```

final_model = Pipeline([
    ('preprocessor', preprocessor),
    (
        'model',
        RandomForestClassifier(
            n_estimators=200,
            max_depth=8,
            min_samples_split=5,
            max_features='sqrt',
            random_state=42
        )
    )
])

final_model.fit(x_train, y_train)

print(
    f"Train : {final_model.score(x_train, y_train):.3f}"
)

print(
    f"Test : {final_model.score(x_test, y_test):.3f}"
)

```

Mais pédagogiquement, je présenterais cette configuration comme **un exemple de configuration testée**, et non comme « la bonne réponse ».

L'intérêt du challenge est justement de faire comprendre que les hyperparamètres doivent être **expérimentés et validés**.

Challenge 2 — Ne pas se limiter à l'accuracy

```

from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score
)

```

```

y_pred = final_model.predict(x_test)

print(
    f'Accuracy : {accuracy_score(y_test, y_pred):.3f}'
)

print(
    f'Precision : {precision_score(y_test, y_pred):.3f}'
)

print(
    f'Recall : {recall_score(y_test, y_pred):.3f}'
)

print(
    f'F1-score : {f1_score(y_test, y_pred):.3f}'
)

```

Matrice de confusion

```

cm = confusion_matrix(y_test, y_pred)

print(cm)

```

On peut également la visualiser :

```

from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=[
        'Non urgent',
        'Urgent'
    ]
).plot()

plt.title('Matrice de confusion')
plt.show()

```

Interprétation métier

La matrice :

	Prédit	
	Non urgent	Urgent
Réel Non urgent	TN	FP
Urgent	FN	TP

Dans notre contexte :

Faux positif

Infrastructure prédite urgente
mais réellement non urgente

Cela peut entraîner une intervention inutile.

Faux négatif

Infrastructure prédite non urgente
mais réellement urgente

C'est potentiellement beaucoup plus problématique dans un contexte de sécurité des infrastructures.

Challenge 3 — Importance des variables

Il faut faire attention à un point important ici :

le `RandomForestClassifier` ne voit pas directement les colonnes originales.

Après `OneHotEncoder`, certaines variables catégorielles deviennent plusieurs colonnes.

On peut récupérer les noms :

```

preprocessor_final = (
    final_model
    .named_steps['preprocessor']
)

feature_names = (
    preprocessor_final
    .get_feature_names_out()
)

importances = (
    final_model
    .named_steps['model']
    .feature_importances_
)

feature_importance = pd.DataFrame({
    'feature': feature_names,
    'importance': importances
})

feature_importance = feature_importance.sort_values(
    'importance',
    ascending=False
)

feature_importance

```

Puis :

```

feature_importance.plot(
    x='feature',
    y='importance',
    kind='barh',
    figsize=(10, 6),
    legend=False
)

plt.title('Importance des variables')
plt.xlabel('Importance')
plt.ylabel('Variable')

```

```
plt.gca().invert_yaxis()
plt.show()
```

Ce que l'on devrait retrouver

Les variables :

```
infrastructure_age
annual_incidents
maintenance_delay
weather_exposure
```

devraient être importantes.

C'est logique puisqu'elles ont directement servi à construire `risk_score`.

En revanche :

```
traffic_level
infrastructure_type
region
```

ne devraient pas avoir une importance comparable, puisqu'elles n'ont pas participé directement à la construction de la cible.

Point pédagogique très intéressant

La forêt peut néanmoins leur attribuer une **petite importance non nulle**.

Pourquoi ?

Parce que le dataset contient du hasard et du bruit.

Cela permet d'introduire une idée importante :

Une importance de variable non nulle ne signifie pas nécessairement qu'il existe une véritable relation métier.

Challenge 4 — Décision métier

Ici, je proposerais aux étudiants de ne surtout pas répondre simplement :

« Oui, le modèle a 90 % d'accuracy donc on peut l'utiliser. »

Une réponse attendue pourrait être :

Le modèle peut constituer un outil d'aide à la décision, mais ses prédictions ne devraient pas automatiquement déclencher des travaux de maintenance lourde sans validation humaine.

Dans ce contexte, les erreurs n'ont pas le même coût. Un faux positif peut provoquer des travaux inutiles et donc générer des coûts financiers et organisationnels. À l'inverse, un faux négatif correspond à une infrastructure considérée comme non urgente alors qu'une intervention serait nécessaire. Pour une infrastructure critique, cette erreur peut avoir des conséquences importantes en matière de sécurité.

L'accuracy seule ne permet donc pas d'évaluer correctement le modèle. Il faut notamment examiner le recall de la classe « urgent », la matrice de confusion et les coûts associés aux différents types d'erreurs.

Enfin, le modèle doit être considéré comme un outil d'aide à la décision. Une validation par les experts métier reste nécessaire avant une décision de maintenance lourde, notamment pour prendre en compte des informations qui peuvent ne pas être présentes dans les données utilisées pour entraîner le modèle.

C'est une excellente occasion d'introduire la notion de **Human-in-the-loop**.

Bonus — « Cassons notre forêt »

La correction est particulièrement intéressante ici.

On construit la forêt B :

```
forest_b = Pipeline([
    ('preprocessor', preprocessor),
    (
        'model',
        RandomForestClassifier(
            n_estimators=100,
            bootstrap=False,
            max_features=None,
            random_state=42
        )
    )
])
```

```

    )
  )
])

forest_b.fit(x_train, y_train)

print(
    f"Train : {forest_b.score(x_train, y_train):.3f}"
)

print(
    f"Test : {forest_b.score(x_test, y_test):.3f}"
)

```

Puis on compare :

```

comparison = pd.DataFrame({
    'model': [
        'Forêt A',
        'Forêt B'
    ],
    'train_score': [
        random_forest.score(x_train, y_train),
        forest_b.score(x_train, y_train)
    ],
    'test_score': [
        random_forest.score(x_test, y_test),
        forest_b.score(x_test, y_test)
    ]
})

comparison

```

Mais surtout : vérifier la diversité

On peut comparer les arbres.

```

forest_a = random_forest.named_steps['model']
forest_b_model = forest_b.named_steps['model']

sample_preprocessed = (

```

```

    random_forest
    .named_steps['preprocessor']
    .transform(x_test.iloc[[0]])
)

predictions_a = [
    tree.predict(sample_preprocessed)[0]
    for tree in forest_a.estimators_
]

predictions_b = [
    tree.predict(sample_preprocessed)[0]
    for tree in forest_b_model.estimators_
]

print('Forêt A :', predictions_a[:20])
print('Forêt B :', predictions_b[:20])

```

Pour aller encore plus loin, on peut vérifier si deux arbres produisent exactement les mêmes prédictions sur tout le jeu de test.

```

test_preprocessed = (
    random_forest
    .named_steps['preprocessor']
    .transform(x_test)
)

predictions_tree_1 = forest_b_model.estimators_[0].predict(
    test_preprocessed
)

predictions_tree_2 = forest_b_model.estimators_[1].predict(
    test_preprocessed
)

print(
    np.mean(predictions_tree_1 == predictions_tree_2)
)

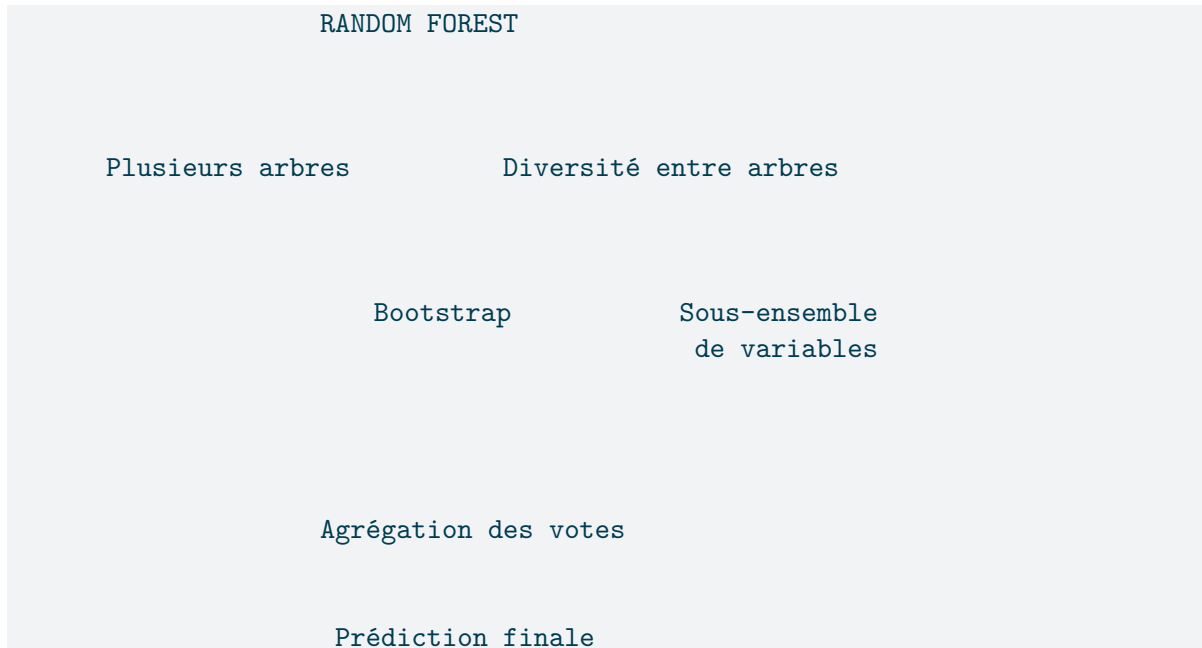
```

Si le résultat est proche de :

cela signifie que les deux arbres produisent quasiment exactement les mêmes prédictions.

Le message pédagogique essentiel du TP

Je terminerais le TP par cette synthèse avec les étudiants :



Et surtout :

Une Random Forest n'est pas simplement un grand nombre d'arbres.

Sa puissance vient de la combinaison de **plusieurs arbres relativement différents**, puis de l'agrégation de leurs prédictions.

C'est précisément ce que ton **bonus « Cassons notre forêt »** permet de faire comprendre expérimentalement.

Une petite amélioration que je ferais au TP

Je rajouterais juste une question juste après le bonus :

Si 100 personnes donnent exactement la même réponse, avons-nous réellement 100 avis indépendants ?

Puis faire le parallèle :

```
100 arbres identiques
    ↓
100 fois la même erreur
    ↓
pas de gain réel

100 arbres différents
    ↓
erreurs différentes
    ↓
le vote peut corriger certaines erreurs
```

C'est, à mon avis, le point conceptuel central à retenir après le passage de Decision Tree à Random Forest.